Criação de uma API **REST**ful com **ASP.NET** e conexão com banco de dados **SQL**.

Pedro Assenção

Problemas

1

Dados isolados em aplicações diferentes

2

Dependência de processos manuais

3

Falta de padronização no acesso aos dados

Objetivo

- Centralizar o acesso às informações
- Automatizar o fluxo de dados entre sistemas
- Padronizar a comunicação entre sistemas
- Reduzir erros no acesso aos dados

FLUXO DA API – SISTEMA ACADÊMICO

01 Banco de dados

Local onde as informações de alunos, cursos e matrículas são armazenadas.

02 Cadastro de Aluno (POST)

A API recebe os dados do aluno no corpo da requisição e os persiste no banco de dados.

03 Matrícula em Curso (POST)

O aluno já cadastrado é vinculado a um curso disponível, criando um registro de matrícula.

04 Consulta de Informações (GET)

A API retorna dados de alunos, cursos ou matrículas conforme o endpoint acessado.

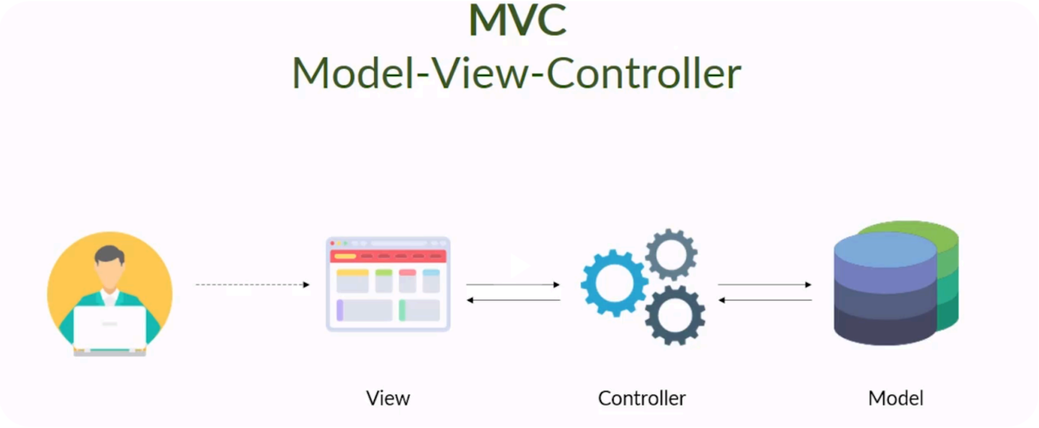
05 Atualização de Dados (PUT)

Informações de um registro existente são alteradas através de uma requisição com os novos dados.

06 Cancelamento de Matrícula (DELETE)

A matrícula é removida do sistema através de uma requisição identificando o registro pelo ID.

Arquitetura (MVC)



View

Interface que o usuário visualiza

(RESPOSTA RETORNADO)

Controller

Responsável por processar as ações do sistema

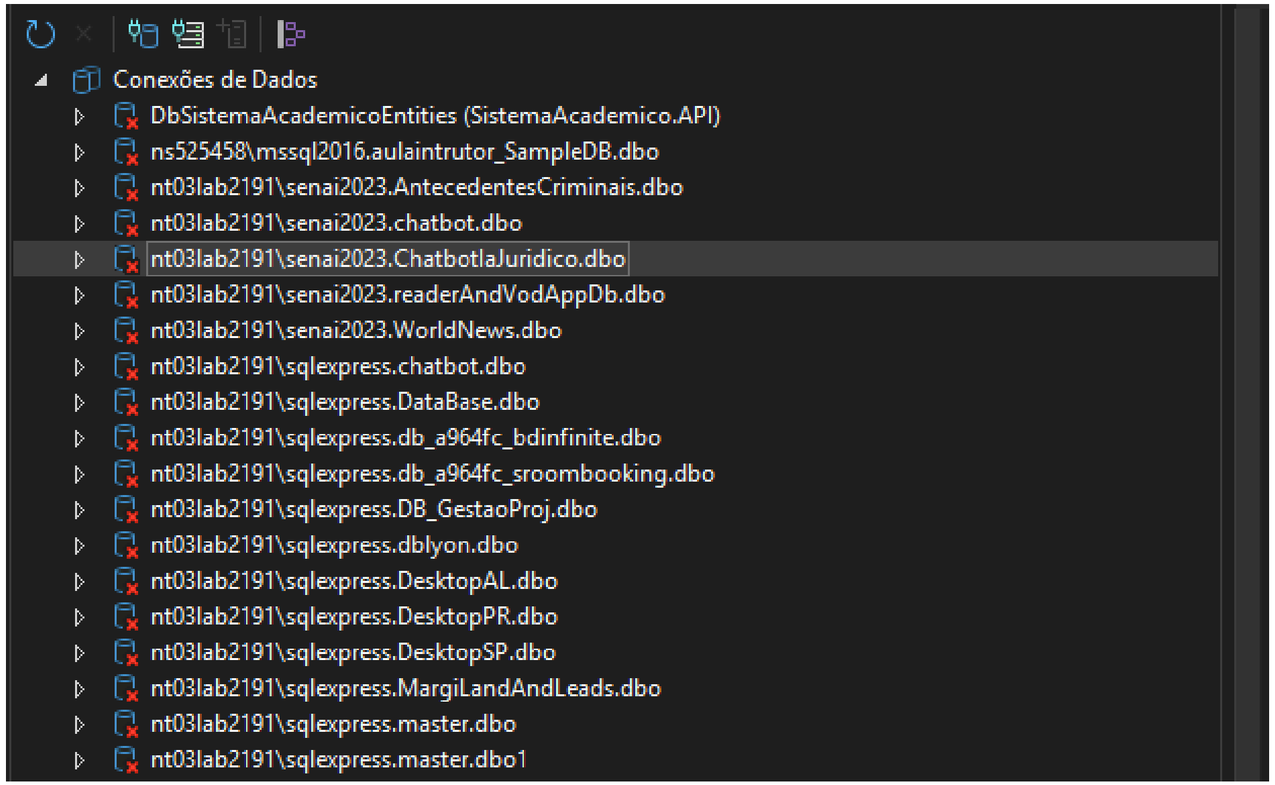
Model

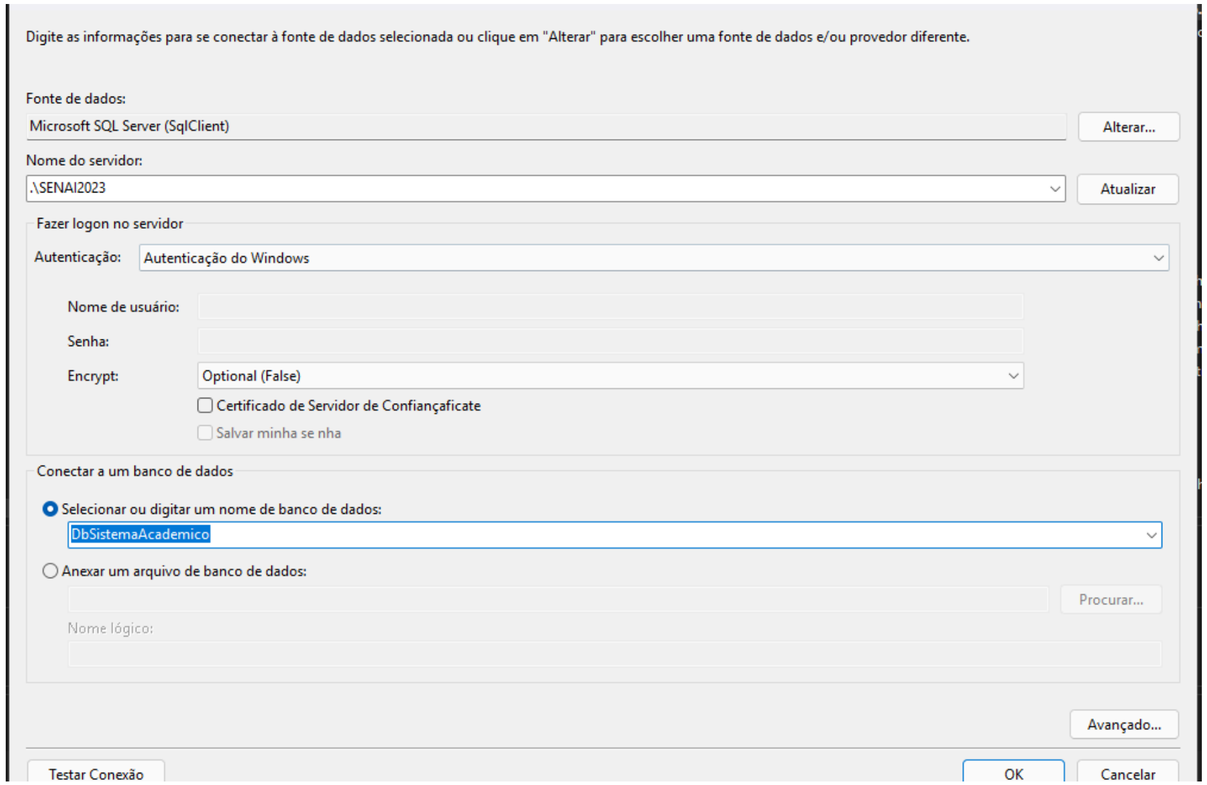
Representa os dados é regras do sistema

Na prática

Conexão entre banco é aplicação

Configurando uma Nova Conexão





Banco de Dados

Aplicação

Tabela: Curso → Classe: Curso (Model)

```sql
CREATE TABLE Curso (
    IdCurso INT IDENTITY(1,1) PRIMARY KEY,
    Nome VARCHAR(100) NOT NULL,
    DuracaoSemestres INT NOT NULL
);

CREATE TABLE Aluno (
    IdAluno INT IDENTITY(1,1) PRIMARY KEY,
    Nome VARCHAR(150) NOT NULL,
    Email VARCHAR(150),
    DataNascimento DATE
);

CREATE TABLE Matricula (
    IdMatricula INT IDENTITY(1,1) PRIMARY KEY,
    IdAluno INT NOT NULL,
    IdCurso INT NOT NULL,
    DataMatricula DATE NOT NULL,
    Status int DEFAULT 0,

    FOREIGN KEY (IdAluno) REFERENCES Aluno(IdAluno),
    FOREIGN KEY (IdCurso) REFERENCES Curso(IdCurso)
);
```

```csharp
namespace SistemaAcademico.API.Models
{
    using System;
    using System.Collections.Generic;

    9 references
    public partial class Curso
    {
        0 references
        public Curso()
        {
            this.Matricula = new HashSet<Matricula>();
        }

        0 references
        public int IdCurso { get; set; }
        0 references
        public string Nome { get; set; }
        0 references
        public int DuracaoSemestres { get; set; }

        1 reference
        public virtual ICollection<Matricula> Matricula { get; set; }
    }
}
```

Operações do sistema - Criar e Consultar (Controller)

Consultar informações existentes

```
// GET: api/curso
[HttpGet]
0 references
public async Task<ActionResult<IEnumerable<Curso>>> GetCursos()
{
    return await _context.Cursos.ToListAsync();
}
```

Criar novos registros

```
// POST: api/curso
[HttpPost]
0 references
public async Task<IActionResult> PostCurso(Curso curso)
{
    try
    {
        _context.Cursos.Add(curso);
        await _context.SaveChangesAsync();

        return Ok();
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Operações do sistema - Atualizar e Remover (Controller)

Atualizar dados existentes

```
// PUT: api/curso/5
[HttpPut("{id}")]
0 references
public async Task<IActionResult> PutCurso(int id, Curso curso)
{
    if (id != curso.IdCurso)
        return BadRequest();

    _context.Entry(curso).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!CursoExists(id))
            return NotFound();
        else
            return BadRequest();
    }

    return Ok();
}
```

Remover registros do sistema

```
// DELETE: api/curso/5
[HttpDelete("{id}")]
0 references
public async Task<IActionResult> DeleteCurso(int id)
{
    var curso = await _context.Cursos.FindAsync(id);

    if (curso == null)
        return NotFound();

    _context.Cursos.Remove(curso);
    await _context.SaveChangesAsync();

    return Ok();
}
```

Visualização (Response)

Pre-Visualização

Responses

Code	Description	Links
200	Success Media type: text/plain Controls Accept header. Example Value | Schema	No links


```
[
  {
    "idCurso": 0,
    "nome": "string",
    "duracaoSemestres": 0,
    "matriculas": [
      {
        "idMatricula": 0,
        "idAluno": 0,
        "idCurso": 0,
        "dataMatricula": "2026-03-20T01:54:56.195Z",
        "status": "string",
        "idAlunoNavigation": {
          "idAluno": 0,
          "nome": "string",
          "email": "string",
          "dataNascimento": "2026-03-20T01:54:56.195Z",
          "matriculas": [
            "string"
          ]
        },
        "idCursoNavigation": "string",
        "boletos": [
          {
            "idBoleto": 0,
            "idMatricula": 0,
            "valor": 0,
            "dataVencimento": "2026-03-20T01:54:56.195Z",
            "status": "string",
            "idMatriculaNavigation": "string"
          }
        ]
      }
    ]
  }
]
```


Response

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:44361/api/Curso' \
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:44361/api/Curso
```

Server response

Code	Details
200	Response body


```
[
  {
    "idCurso": 1,
    "nome": "Engenharia de Software -123",
    "duracaoSemestres": 8,
    "matriculas": []
  },
  {
    "idCurso": 2,
    "nome": "Administração",
    "duracaoSemestres": 8,
    "matriculas": []
  },
  {
    "idCurso": 3,
    "nome": "Direito",
    "duracaoSemestres": 10,
    "matriculas": []
  }
]
```


Response headers

```
content-type: application/json; charset=utf-8
date: Fri,20 Mar 2026 01:56:19 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code	Description
200	

REFERÊNCIAS DA AULA

1. https://learn.microsoft.com/pt-br/aspnet/core/data/ef-mvc/crud?view=aspnetcore-10.0
2. https://learn.microsoft.com/pt-br/aspnet/core/data/ef-mvc/migrations?view=aspnetcore-10.0
3. https://learn.microsoft.com/pt-br/aspnet/core/data/ef-mvc/read-related-data?view=aspnetcore-10.0
4. https://marcionizzola.medium.com/criando-um-simples-crud-com-net-mvc-com-dotnet-6-917cf723815e
5. https://learn.microsoft.com/pt-br/aspnet/core/tutorials/first-web-api?view=aspnetcore-10.0&tabs=visual-studio